

English-Hindi Neural Machine Translation using Transformer

UJJAWAL MALVIYA

May 31, 2026

Abstract

Natural Language Processing (NLP) is one of the most important domains of Artificial Intelligence. Machine Translation enables automatic translation of text between languages. This project presents an English-to-Hindi Neural Machine Translation system using a Transformer-based architecture. The IIT Bombay English-Hindi Parallel Corpus is utilized for training and evaluation. A MarianMT Transformer model is employed to generate Hindi translations from English input sentences. A Streamlit-based web application is also developed to provide an interactive translation interface. The project demonstrates the effectiveness of Transformer models in solving real-world language translation problems.

Contents

1	Introduction	3
2	Objectives	3
3	Problem Statement	3
4	Dataset Description	3
4.1	Dataset Used	3
4.2	Dataset Components	4
5	Literature Review	4
6	Methodology	4
6.1	Data Preprocessing	4
6.2	Tokenization	5
7	Transformer Architecture	5
7.1	Encoder	5
7.2	Decoder	5
7.3	Attention Mechanism	5
8	Tools and Technologies Used	6

9	Implementation	6
9.1	Model Selection	6
9.2	Training Process	6
9.3	Translation Module	6
10	Source Code	7
10.1	Training Script (train.py)	7
10.2	Translation Script (translate.py)	9
10.3	Streamlit Application (app.py)	10
11	Output	11
11.1	Application Interface	11
12	System Architecture	12
13	Results	12
14	Advantages	12
15	Limitations	12
16	Future Scope	13
17	Conclusion	13
18	References	13

1 Introduction

Natural Language Processing (NLP) is a branch of Artificial Intelligence that focuses on enabling computers to understand and generate human language.

Machine Translation is an important NLP application that automatically translates text from one language into another. Traditional translation systems relied on rule-based approaches, whereas modern systems utilize deep learning models.

The Transformer architecture has become the standard approach for machine translation because of its ability to learn contextual relationships between words and process long sequences efficiently.

This project develops an English-to-Hindi translation system using Transformer-based Neural Machine Translation techniques.

2 Objectives

The objectives of this project are:

- To understand Natural Language Processing concepts.
- To study Transformer-based architectures.
- To build an English-to-Hindi translation system.
- To utilize the IIT Bombay English-Hindi Parallel Corpus.
- To evaluate translation performance.
- To develop a user-friendly translation interface.

3 Problem Statement

Language barriers often make communication difficult between people speaking different languages.

The objective of this project is to design and implement an automated translation system capable of translating English text into Hindi using Transformer-based Neural Machine Translation.

4 Dataset Description

4.1 Dataset Used

The dataset used in this project is the IIT Bombay English-Hindi Parallel Corpus.

The corpus contains parallel sentence pairs:

- English Sentence
- Corresponding Hindi Sentence

Example:

English	Hindi (Transliteration)
Hello	Namaste
How are you?	Aap Kaise Hain
Good Morning	Suprabhat
I Love India	Mujhe Bharat Pasand Hai

Table 1: Sample Parallel Corpus

4.2 Dataset Components

The dataset contains:

- Training Set
- Validation (Development) Set
- Test Set

5 Literature Review

Neural Machine Translation has evolved significantly over the past decade.

Major developments include:

- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory Networks (LSTMs)
- Attention Mechanisms
- Transformer Architecture

The Transformer architecture introduced in the paper "Attention Is All You Need" became the foundation of modern machine translation systems.

6 Methodology

The overall workflow is:

Dataset → Preprocessing → Tokenization → Transformer Model → Translation
Generation

6.1 Data Preprocessing

The following preprocessing steps are performed:

- Removal of empty entries
- Removal of unnecessary spaces
- Sentence alignment verification
- Data formatting

6.2 Tokenization

Tokenization converts text into smaller units called tokens.

Example:

Input Sentence:

I love India

Tokens:

[I] [love] [India]

The Transformer model processes tokenized input rather than raw text.

7 Transformer Architecture

The Transformer consists of two primary components:

7.1 Encoder

The encoder processes the English input sentence and creates contextual representations.

Example:

Input:

I Love India

7.2 Decoder

The decoder generates the translated Hindi sentence token-by-token.

Example:

Output: Mujhe Bharat Pasand Hai

7.3 Attention Mechanism

Attention enables the model to focus on important words during translation.

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where:

- Q = Query
- K = Key
- V = Value
- d_k = Dimension of Key Vectors

8 Tools and Technologies Used

Technology	Purpose
Python	Used for implementing the complete translation system and data processing.
Transformers Library	Provides pre-trained Transformer models such as MarianMT for machine translation.
PyTorch	Deep learning framework used as backend for training and inference.
Streamlit	Used to create an interactive web interface for translation.
IIT Bombay English-Hindi Corpus	Dataset containing parallel English-Hindi sentence pairs used for training and evaluation.
Hugging Face	Provides access to pre-trained NLP models and tokenizers.

9 Implementation

9.1 Model Selection

The MarianMT Transformer model is used:

`Helsinki-NLP/opus-mt-en-hi`

9.2 Training Process

Training steps:

1. Load dataset.
2. Perform preprocessing.
3. Tokenize sentences.
4. Fine-tune Transformer model.
5. Save trained model.

9.3 Translation Module

Translation process:

1. Accept user input.
2. Tokenize input sentence.
3. Generate translation.
4. Decode generated tokens.
5. Display Hindi translation.

10 Source Code

10.1 Training Script (train.py)

```
from datasets import Dataset
from transformers import (
    MarianMTModel,
    MarianTokenizer,
    Seq2SeqTrainer,
    Seq2SeqTrainingArguments,
    DataCollatorForSeq2Seq
)\\
# Load Dataset

with open(
    "D:/EduExpose/inter/dataset/IITB.en-hi.en",
    "r",
    encoding="utf-8"
) as f:
    english_sentences = f.readlines()

with open(
    "D:/EduExpose/inter/dataset/IITB.en-hi.hi",
    "r",
    encoding="utf-8"
) as f:
    hindi_sentences = f.readlines()
english_sentences = [x.strip() for x in english_sentences]
hindi_sentences = [x.strip() for x in hindi_sentences]

english_sentences = english_sentences[:100]
hindi_sentences = hindi_sentences[:100]

print("Training Samples:", len(english_sentences))
# Create Dataset
data = {
    "english": english_sentences,
    "hindi": hindi_sentences
}

dataset = Dataset.from_dict(data)

# Train/Validation Split
dataset = dataset.train_test_split(
    test_size=0.1,
    seed=42
)
```

```

train_dataset = dataset["train"]
eval_dataset = dataset["test"]
#load model

model_name = "Helsinki-NLP/opus-mt-en-hi"

tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)

#tokenization

def preprocess(examples):

    model_inputs = tokenizer(
        examples["english"],
        max_length=128,
        truncation=True
    )

    labels = tokenizer(
        examples["hindi"],
        max_length=128,
        truncation=True
    )

    model_inputs["labels"] = labels["input_ids"]

    return model_inputs

train_dataset = train_dataset.map(
    preprocess,
    batched=True
)

eval_dataset = eval_dataset.map(
    preprocess,
    batched=True
)

# Remove text columns
train_dataset = train_dataset.remove_columns(
    ["english", "hindi"]
)

eval_dataset = eval_dataset.remove_columns(
    ["english", "hindi"]
)

```

```

data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model
)
# Training Arguments

training_args = Seq2SeqTrainingArguments(
    output_dir="./model",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    num_train_epochs=1,
    weight_decay=0.01,
    predict_with_generate=True,
    logging_steps=50,
    report_to="none"
)
# Trainer
trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    data_collator=data_collator
)
trainer.train()
model.save_pretrained("./model")
tokenizer.save_pretrained("./model")

print("Training Completed Successfully!")

```

10.2 Translation Script (translate.py)

```

from transformers import MarianMTModel, MarianTokenizer

# Load saved model
model_path = "./model"

tokenizer = MarianTokenizer.from_pretrained(model_path)
model = MarianMTModel.from_pretrained(model_path)

# Translation function
def translate_text(text):

    inputs = tokenizer(text, return_tensors="pt")

    translated = model.generate(**inputs)

```

```

        output = tokenizer.decode(
            translated[0],
            skip_special_tokens=True
        )

    return output

# User input
text = input("Enter English Sentence: ")

translation = translate_text(text)

print("Hindi Translation:", translation)

```

10.3 Streamlit Application (app.py)

```

import streamlit as st
from transformers import MarianMTModel, MarianTokenizer

# Load model
model_path = "./model"

tokenizer = MarianTokenizer.from_pretrained(model_path)
model = MarianMTModel.from_pretrained(model_path)

# Translation function
def translate_text(text):

    inputs = tokenizer(text, return_tensors="pt")

    translated = model.generate(**inputs)

    output = tokenizer.decode(
        translated[0],
        skip_special_tokens=True
    )

    return output

# Streamlit UI
st.title("English to Hindi Translator")

user_input = st.text_input("Enter English Text")

if st.button("Translate"):

    result = translate_text(user_input)

```

```
st.success(result)
```

11 Output

11.1 Application Interface

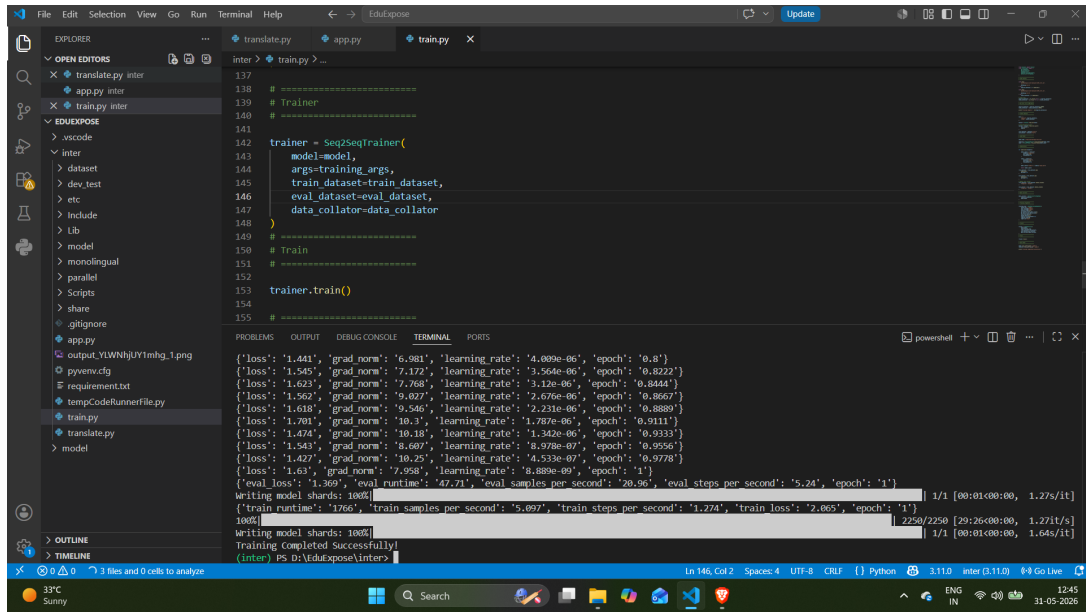


Figure 1: Training completed successfully

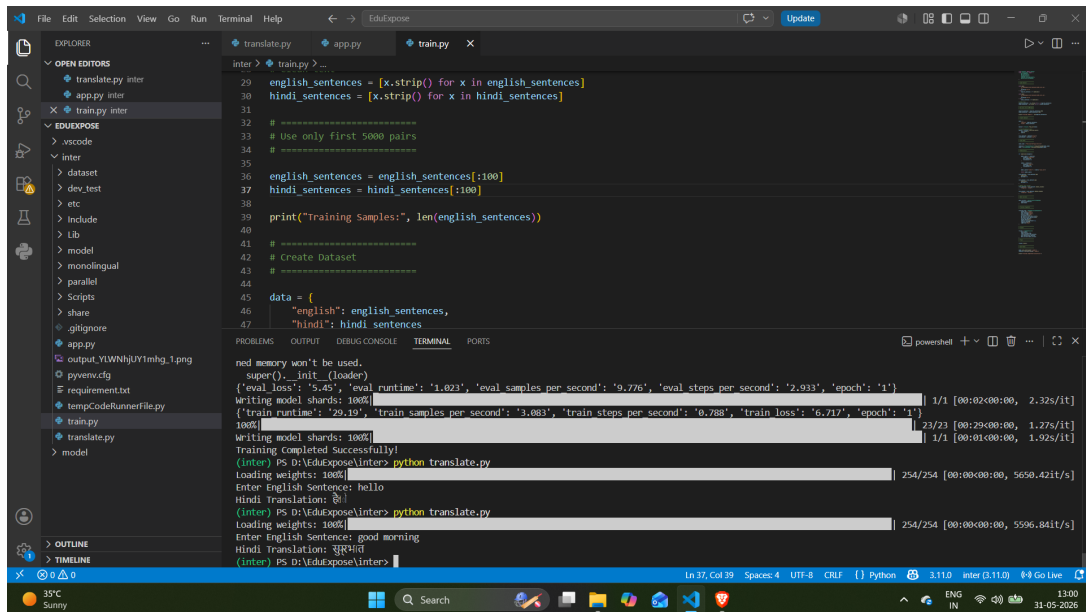


Figure 2: Translate: English-Hindi

12 System Architecture

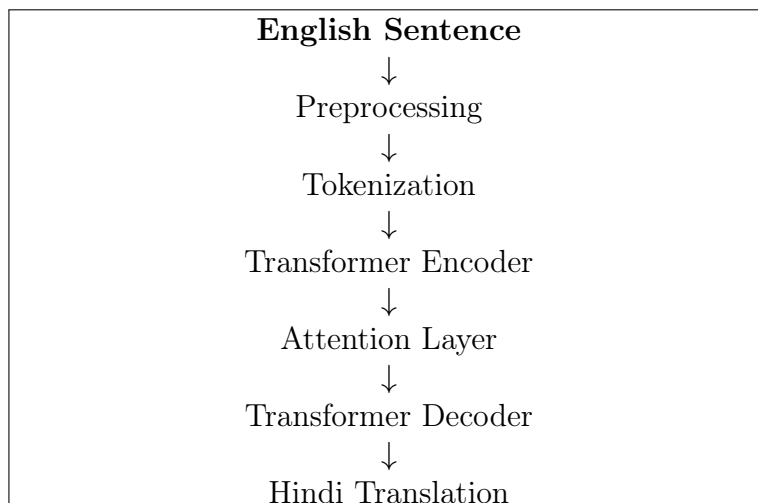


Figure 3: System Architecture of English-Hindi Translation System

13 Results

Sample translation outputs:

English Input	Hindi Output (Transliteration)
Hello	Namaste
How are you?	Aap Kaise Hain
Good Morning	Suprabhat
I Love India	Mujhe Bharat Pasand Hai

Table 2: Sample Translation Results

14 Advantages

- High translation quality.
- Context-aware translation.
- Fast inference.
- Scalable architecture.
- User-friendly interface.

15 Limitations

- Requires large datasets.

- Computationally intensive training.
- Sensitive to data quality.
- Rare linguistic patterns may reduce accuracy.

16 Future Scope

Future enhancements may include:

- Multi-language translation.
- Voice-based translation.
- Mobile application deployment.
- Cloud deployment.
- Larger-scale fine-tuning.
- Speech-to-text integration.

17 Conclusion

This project successfully demonstrates an English-to-Hindi Neural Machine Translation system using Transformer architecture. The IIT Bombay English-Hindi Parallel Corpus was utilized for experimentation and evaluation. The MarianMT model effectively translated English sentences into Hindi. The developed Streamlit interface further improved usability by providing an interactive platform for users.

18 References

1. Vaswani, A. et al., "Attention Is All You Need", NeurIPS 2017.
2. IIT Bombay English-Hindi Parallel Corpus.
3. Hugging Face Transformers Documentation.
4. PyTorch Documentation.
5. MarianMT Documentation.